

欠拟合 (Underfitting) 原因分析及解决方案

一、什么是欠拟合？

定义： 模型在训练集和测试集上表现都很差的现象。

本质： 模型过于简单，无法捕捉数据中的基本规律和模式。

代码块

- 1 训练误差：很高 ✗
- 2 测试误差：很高 ✗
- 3 泛化能力：很差 ✗
- 4 学习能力：不足 ✗

对比过拟合：

- 过拟合：训练好，测试差（学得太细）
- 欠拟合：训练差，测试差（学得太浅）

二、🔍 产生欠拟合的原因

1 数据层面原因

原因	说明	影响
数据质量差	噪声太多、标签错误	模型无法学习有效模式

特征信息不足	缺少关键特征	模型缺乏足够的输入信息
数据分布复杂	非线性关系强	线性模型无法拟合
数据预处理不当	未标准化/归一化	特征尺度差异大，影响学习

2 模型层面原因

原因	说明	影响
模型过于简单	参数太少、层数太浅	模型容量不足以表达数据规律
模型选择不当	用简单模型解决复杂问题	小马拉大车
激活函数不当	使用线性激活函数	无法学习非线性关系
缺乏非线性能力	线性模型处理非线性问题	拟合能力受限

3 训练层面原因

原因	说明	影响
训练时间不足	迭代次数太少	模型还没学会就停止了
学习率过低	参数更新太慢	收敛速度极慢或陷入局部最优
优化器选择不当	使用了不合适的优化算法	训练效率低下
批量大小过大	每次更新基于大量样本	梯度估计过于平滑，学习缓慢

三、 检测欠拟合的方法

1. 学习曲线分析

代码块

```
1  from sklearn.model_selection import learning_curve
2  import matplotlib.pyplot as plt
3
4  train_sizes, train_scores, val_scores = learning_curve(
5      model, X, y, cv=5, train_sizes=np.linspace(0.1, 1.0, 10)
6  )
7
8  plt.plot(train_sizes, train_scores.mean(axis=1), label='Training score')
9  plt.plot(train_sizes, val_scores.mean(axis=1), label='Validation score')
10 plt.legend()
11 plt.show()
```

欠拟合特征：

- 训练分数和验证分数都很低
- 两条曲线都很低且接近
- 增加训练数据，分数没有明显提升

2. 误差对比法

代码块

```
1  # 训练集误差 vs 测试集误差
2  train_error = model.score(X_train, y_train)
3  test_error = model.score(X_test, y_test)
4
5  if train_error < 0.60 and test_error < 0.60:
6      print("⚠️ 可能欠拟合！")
```

判断标准：

- 训练准确率 < 60%，测试准确率 < 60% → 严重欠拟合
 - 两者都低且接近 → 欠拟合
 - 训练低，测试更低 → 可能同时存在欠拟合和过拟合
-

四、解决欠拟合的方法（大全）

第一类：模型层面解决方案

1. 增加模型复杂度 最核心方法

决策树：

代码块

```
1  from sklearn.tree import DecisionTreeClassifier
2
3  # 增加模型复杂度
4  tree = DecisionTreeClassifier(
5      max_depth=None,          # 不限制深度
6      min_samples_split=2,     # 降低分割阈值
7      min_samples_leaf=1      # 允许更小的叶子节点
8  )
```

神经网络：

代码块

```
1  import tensorflow as tf
2
3  # 增加网络层数和神经元数量
```

```
4 model = tf.keras.Sequential([
5     tf.keras.layers.Dense(256, activation='relu'), # 增加神经元
6     tf.keras.layers.Dense(128, activation='relu'), # 增加层数
7     tf.keras.layers.Dense(64, activation='relu'),
8     tf.keras.layers.Dense(10, activation='softmax')
9 ])
```

2. 选择更强大的模型

代码块

```
1 # 从简单模型升级到复杂模型
2 from sklearn.linear_model import LogisticRegression
3 from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
4
5 # 简单模型 → 复杂模型
6 # 逻辑回归 → 随机森林 → XGBoost/LightGBM
7
8 # 使用集成方法
9 from sklearn.ensemble import VotingClassifier
10
11 model1 = LogisticRegression()
12 model2 = RandomForestClassifier()
13 model3 = GradientBoostingClassifier()
14
15 ensemble = VotingClassifier(
16     estimators=[('lr', model1), ('rf', model2), ('gb', model3)],
17     voting='soft'
18 )
```

第二类：特征层面解决方案

3. 增加特征数量

代码块

```
1  # 1. 添加新特征
2  df['new_feature'] = df['feature1'] * df['feature2'] # 特征交叉
3  df['feature_squared'] = df['feature1'] ** 2        # 多项式特征
4
5  # 2. 多项式特征
6  from sklearn.preprocessing import PolynomialFeatures
7  poly = PolynomialFeatures(degree=2, include_bias=False)
8  X_poly = poly.fit_transform(X)
9
10 # 3. 特征编码 (类别特征)
11 from sklearn.preprocessing import OneHotEncoder
12 encoder = OneHotEncoder(sparse=False)
13 X_encoded = encoder.fit_transform(categorical_features)
14
15 # 4. 特征组合
16 from sklearn.preprocessing import InteractionFeatures
17 # 自定义特征组合逻辑
```

4. 特征工程优化

代码块

```
1  # 1. 分箱/离散化
2  from sklearn.preprocessing import KBinsDiscretizer
3  discretizer = KBinsDiscretizer(n_bins=5, encode='onehot-dense',
4  strategy='uniform')
5  X_binned = discretizer.fit_transform(X)
6
7  # 2. 对数变换 (处理偏态分布)
8  import numpy as np
9  X_log = np.log1p(X)
10
11 # 3. 标准化/归一化
12 from sklearn.preprocessing import StandardScaler, MinMaxScaler
13 scaler = StandardScaler()
14 X_scaled = scaler.fit_transform(X)
```

```
14
15 # 4. 特征选择 (保留重要特征)
16 from sklearn.feature_selection import SelectKBest, f_classif
17 selector = SelectKBest(score_func=f_classif, k='all')
18 selector.fit(X, y)
19 feature_scores = selector.scores_
```

第三类：训练策略解决方案

5. 增加训练时间

代码块

```
1 # 1. 增加迭代次数 (epochs)
2 model.fit(X_train, y_train, epochs=100) # 原来是 10
3
4 # 2. 减少早停耐心值
5 early_stopping = tf.keras.callbacks.EarlyStopping(
6     monitor='val_loss',
7     patience=20, # 原来是 5
8     restore_best_weights=True
9 )
10
11 # 3. 检查学习曲线, 确保充分训练
```

6. 调整学习率

代码块

```
1 # 1. 增大学习率
2 from sklearn.linear_model import SGDClassifier
```

```

3  sgd = SGDClassifier(learning_rate='constant', eta0=0.01) # 增大初始学习率
4
5  # 2. 使用学习率调度器
6  import tensorflow as tf
7  lr_scheduler = tf.keras.callbacks.LearningRateScheduler(
8      lambda epoch: 0.001 * (1.5 ** epoch) # 逐步增大学习率
9  )
10
11 # 3. 更换优化器
12 optimizer = tf.keras.optimizers.Adam(learning_rate=0.01) # 增大学习率
13 model.compile(optimizer=optimizer, loss='categorical_crossentropy')

```

7. 调整批量大小

代码块

```

1  # 减小批量大小, 增加更新频率
2  model.fit(X_train, y_train,
3            batch_size=16, # 原来是 128
4            epochs=100)
5
6  # 小批量有助于跳出局部最优

```

第四类：数据层面解决方案

8. 改善数据质量

代码块

```

1  # 1. 处理缺失值
2  from sklearn.impute import SimpleImputer

```



```
3 imputer = SimpleImputer(strategy='mean') # 或'median', 'most_frequent'
4 X_imputed = imputer.fit_transform(X)
5
6 # 2. 去除异常值
7 from sklearn.ensemble import IsolationForest
8 iso_forest = IsolationForest(contamination=0.05)
9 outliers = iso_forest.fit_predict(X)
10 X_clean = X[outliers == 1]
11
12 # 3. 处理类别不平衡
13 from imblearn.over_sampling import SMOTE
14 smote = SMOTE()
15 X_resampled, y_resampled = smote.fit_resample(X, y)
```

9. 特征重要性分析

代码块

```
1 # 1. 使用随机森林评估特征重要性
2 from sklearn.ensemble import RandomForestClassifier
3 rf = RandomForestClassifier()
4 rf.fit(X, y)
5 importances = rf.feature_importances_
6
7 # 2. 使用 SHAP 值
8 import shap
9 explainer = shap.TreeExplainer(rf)
10 shap_values = explainer.shap_values(X)
11
12 # 3. 添加领域知识特征
13 # 根据业务理解添加有意义的特征
```

五、解决方案对比总结

方法	适用场景	效果	实施难度	推荐指数
增加模型复杂度	模型太简单	★★★★★	低	★★★★★★
增加特征	特征不足	★★★★★★	中	★★★★★★
多项式特征	非线性关系	★★★★★	低	★★★★★
调整学习率	训练缓慢	★★★★★	低	★★★★★
增加训练时间	训练不足	★★★★★	低	★★★★★
更换优化器	优化效率低	★★★★★	低	★★★★★
改善数据质量	数据问题	★★★★	中	★★★★
特征工程	特征质量差	★★★★★	高	★★★★★
集成学习	单模型能力不足	★★★★★★	中	★★★★★★
减小批量大小	学习缓慢	★★★★	低	★★★★

六、🎯 实战建议与决策流程

解决欠拟合的决策树：

代码块

```
1  发现欠拟合？
2  |
3  — 训练误差很高？ → 增加模型复杂度
4  |
5  — 特征很少？ → 增加特征/特征工程
6  |
7  — 训练很快停止？ → 增加训练时间/调整学习率
8  |
9  — 数据质量差？ → 数据清洗/预处理
```

10 |
11 └─ 线性模型? → 换非线性模型/加多项式特征

优先尝试顺序:

代码块

- 1 第一步：模型调整
- 2 ✓ 增加模型复杂度（层数、神经元、深度）
- 3 ✓ 更换更强大的模型
- 4 ✓ 使用非线性激活函数
- 5
- 6 第二步：特征增强
- 7 ✓ 添加新特征
- 8 ✓ 多项式特征
- 9 ✓ 特征交叉
- 10 ✓ 特征编码
- 11
- 12 第三步：训练优化
- 13 ✓ 增加训练轮数
- 14 ✓ 调整学习率
- 15 ✓ 减小批量大小
- 16 ✓ 更换优化器
- 17
- 18 第四步：数据改善
- 19 ✓ 数据清洗
- 20 ✓ 处理异常值
- 21 ✓ 特征工程
- 22 ✓ 增加数据量

七、 记忆小技巧

 记忆口诀：

代码块

- 1 欠拟合，太简单，学不会来记不清。
- 2 加复杂，增特征，调学习率延训练。
- 3 多项式，换模型，数据清洗保质量。

 核心要点记忆：

代码块

- 1 原因：模型简、特征少、训练短、学习慢
- 2 解决：加复杂、增特征、调参数、延训练
- 3 核心：提高模型表达能力，增强学习能力

 类比记忆：

欠拟合现象	类比	解决方法
学不会基本规律	小学生做高数题	提升能力（增加复杂度）
信息不足	盲人摸象	增加信息（添加特征）
训练不足	练习时间不够	加强训练（增加时间）
学习太慢	步伐太小	加快步伐（调整学习率）

八、 代码示例：完整解决方案

代码块

```
1  import numpy as np
2  import pandas as pd
3  from sklearn.model_selection import train_test_split, learning_curve
4  from sklearn.preprocessing import PolynomialFeatures, StandardScaler
5  from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
6  from sklearn.pipeline import Pipeline
7  import matplotlib.pyplot as plt
8
9  # 1. 数据准备
10 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
11 random_state=42)
12
13 # 2. 检查当前模型表现 (基线)
14 from sklearn.linear_model import LogisticRegression
15 baseline_model = LogisticRegression()
16 baseline_model.fit(X_train, y_train)
17 train_score = baseline_model.score(X_train, y_train)
18 test_score = baseline_model.score(X_test, y_test)
19
20 print(f"基线模型 - 训练得分: {train_score:.4f}, 测试得分: {test_score:.4f}")
21
22 if train_score < 0.7 and test_score < 0.7:
23     print("⚠ 检测到欠拟合, 开始优化...")
24
25 # 3. 方案一: 增加模型复杂度
26 complex_model = RandomForestClassifier(
27     n_estimators=100,
28     max_depth=None,
29     min_samples_split=2
30 )
31
32 # 4. 方案二: 添加多项式特征
33 pipeline = Pipeline([
34     ('poly', PolynomialFeatures(degree=2, include_bias=False)),
35     ('scaler', StandardScaler()),
36     ('model', GradientBoostingClassifier(n_estimators=100))
37 ])
38
39 # 5. 训练优化后的模型
40 pipeline.fit(X_train, y_train)
41
42 # 6. 评估结果
```

```
42     new_train_score = pipeline.score(X_train, y_train)
43     new_test_score = pipeline.score(X_test, y_test)
44
45     print(f"优化模型 - 训练得分: {new_train_score:.4f}, 测试得分:
46           {new_test_score:.4f}")
47
48     # 7. 学习曲线验证
49     train_sizes, train_scores, val_scores = learning_curve(
50         pipeline, X_train, y_train, cv=5,
51         train_sizes=np.linspace(0.1, 1.0, 10)
52     )
53
54     plt.plot(train_sizes, train_scores.mean(axis=1), label='Training score')
55     plt.plot(train_sizes, val_scores.mean(axis=1), label='Validation score')
56     plt.legend()
57     plt.title('Learning Curve After Optimization')
58     plt.show()
```

九、 总结

核心要点：

1. 欠拟合本质：模型复杂度 < 数据规律复杂度
2. 最常用方法：增加模型复杂度、添加特征、调整训练参数
3. 最佳实践：从简单开始，逐步增加复杂度直到合适
4. 关键指标：训练误差和测试误差都很高

黄金法则：

代码块

```
1     "复杂度不足则增加，特征不够则添加"
```

- 2

"训练不足则延长，学习太慢则加速"
- 3

"欠拟合是能力问题，过拟合是控制问题"

 实战建议：

- 先建立简单基线模型
- 逐步增加复杂度，监控训练和验证表现
- **模型复杂度和特征质量是解决欠拟合的关键**
- 不要一开始就用最复杂的模型，循序渐进

与过拟合的对比：

问题	训练误差	测试误差	核心原因	解决方向
欠拟合	高	高	模型太简单	增加复杂度
过拟合	低	高	模型太复杂	降低复杂度
理想状态	低	低	复杂度适中	保持平衡

记住：解决欠拟合的关键是提高模型的表达能力和学习能力！  